SOC-Triage-Gym Hackathon Guide

1. What This Project Is

SOC-Triage-Gym is an OpenEnv-compatible reinforcement learning environment that simulates the workflow of a Security Operations Center analyst. The environment is designed so an agent does not simply classify a static record. Instead, it must investigate alerts, gather evidence, correlate related activity, decide whether an alert is a true positive, false positive, or benign true positive, map findings to MITRE ATT&CK, recommend containment, and then submit the investigation for grading.

The core value of the project is that it models security investigation as an interactive reasoning task rather than a one-shot prediction task. That is what makes it relevant for a hackathon around realistic agent environments.

2. Why This Matters For The Hackathon

If someone asks why this project is important, the short answer is:

- most environments are narrow and toy-like
- real SOC work is noisy, sequential, and high stakes
- this project creates a realistic training loop for that type of work

The strongest framing for judges is:

- this is not just another chatbot wrapper
- this is a structured environment with hidden ground truth, deterministic seeds, explicit tool interfaces, measurable rewards, and reproducible evaluation
- it supports both learned agents and heuristic baselines

3. Project Goals

The project tries to satisfy several goals at once:

- realistic SOC investigation behavior
- deterministic reproducibility through seeded scenario generation
- OpenEnv compliance for easy validation and deployment
- rich enough action and observation spaces for agent training
- multiple difficulty levels instead of a single benchmark task
- explicit grading so outcomes are measurable and comparable

4. Current Submission Status

As of the current repo state:

- local OpenEnv validation passes
- targeted pytest validation passes
- the Hugging Face Space has already been pushed
- the baseline endpoint is wired to a deterministic heuristic agent
- the benchmark covers all 4 tasks

Current measured deterministic baseline for `seed=42`:

- Phishing: `0.65`
- Lateral Movement: `0.71`
- Queue Management: `0.885`
- Insider Threat: `0.92`
- Overall average: `0.791`

These numbers are useful in the hackathon because they show:

- the environment is not broken
- the baseline is meaningful
- hard tasks are tractable

- the repo produces reproducible results

5. High-Level Architecture

The project has five major layers:

1. Contract layer `models.py` Defines the public and internal Pydantic models.
2. Runtime layer `server/app.py` and `server/environment.py` Exposes the HTTP API and implements the environment state machine.
3. Scenario layer `scenarios/*.py` Generates deterministic tasks and hidden ground truth.
4. Evaluation layer `graders/*.py` Scores the final investigation state.
5. Agent and proof layer `baseline_agent.py`, `inference.py`, `benchmark.py`, `tests/*` Provides a deterministic baseline, an inference entrypoint, benchmarking, and tests.

6. Important Files And What They Do

Root files

- `README.md` Public-facing summary of the project.
- `openenv.yaml` OpenEnv manifest describing tasks, action space, observation space, and metadata.
- `models.py` Shared Pydantic schema definitions and enums.
- `baseline_agent.py` Deterministic heuristic baseline agent.
- `inference.py` Inference runner that can use either an LLM or the deterministic baseline.
- `benchmark.py` Multi-task reproducibility benchmark using the `/baseline` endpoint.
- `pyproject.toml` Package metadata and entry points.
- `Dockerfile` Containerized deployment path.

- `pytest.ini` Test configuration.

Server files

- `server/app.py` FastAPI application, API endpoints, MCP endpoint, REST helpers, and baseline endpoint.
- `server/environment.py` Core environment state machine and reward logic.
- `server/ui.py` Browser-based UI payload.

Scenario files

- `scenarios/phishing.py` Easy single-alert task.
- `scenarios/lateral_movement.py` Five-alert kill chain task.
- `scenarios/queue_management.py` Twenty-alert mixed queue with noise.
- `scenarios/insider_threat.py` Thirty-alert expert task with three hidden chains.
- `scenarios/base.py` Common seeded generators and helper methods.

Grader files

- `graders/base.py` Shared grading helpers.
- `graders/phishing_grader.py`
- `graders/lateral_movement_grader.py`
- `graders/queue_management_grader.py`
- `graders/insider_threat_grader.py`

Tool simulation files

- `tools/enrichment.py` IOC enrichment logic.
- `tools/log_query.py` Simulated log retrieval.
- `tools/correlation.py` Correlation logic between alerts.
- `tools/asset_lookup.py` Asset inventory lookup.

- `tools/user_lookup.py` User directory lookup.

Test files

- `tests/test_server.py` FastAPI endpoint behavior.
- `tests/test_environment.py` Environment transitions and budget behavior.
- `tests/test_scenarios.py` Determinism and ground-truth completeness.
- `tests/test_graders.py` Scoring logic.
- `tests/conftest.py` Fixtures.

7. Core Data Model

The central contract lives in `models.py`.

Core enums

- `AlertSeverity` `critical`, `high`, `medium`, `low`, `info`
- `AlertClassification` `true_positive`, `false_positive`, `benign_true_positive`, `unclassified`
- `IndicatorType` `ip`, `domain`, `file_hash`, `email`, `url`, `user`
- `LogSource` `firewall`, `proxy`, `dns`, `endpoint`, `auth`, `email_gateway`, `ids`, `cloud_trail`
- `ResponseActionType` `isolate_endpoint`, `disable_account`, `block_ip`, `block_domain`, `quarantine_file`, `reset_password`, `revoke_sessions`, `no_action`
- `ActionType` The actual action names the environment accepts.

Public agent-facing models

- `SOCAction` Flat action object. The `action_type` field determines which optional fields matter.
- `SOCObservation` Full environment observation after `reset` or `step`.

- `EnvironmentState` Lightweight state metadata returned by `/state`.

Key sub-models

- `AlertMeta` Alert metadata visible to the agent.
- `EnrichmentResult` Threat intel result for an indicator.
- `LogEntry` One event returned from a log source.
- `CorrelatedEvent` Evidence that two or more alerts are linked.
- `AssetInfo` Simulated asset inventory response.
- `UserInfo` Simulated identity/profile response.
- `InvestigationState` The main per-alert record that graders inspect.

Hidden internal models

- `GroundTruth` Answer key for the task. Never exposed to the agent.
- `ScenarioConfig` Full task data, including ground truth and all backing databases.

8. HTTP API Surface

The main server lives in `server/app.py`.

Core OpenEnv-style endpoints

- `GET /health` Liveness endpoint.
- `GET /metadata` Environment metadata.
- `GET /schema` Pydantic JSON schemas for action, observation, and state.
- `POST /reset` Starts a new episode.
- `POST /step` Executes one action.
- `GET /state` Returns current metadata without consuming a step.

Additional endpoints

- `POST /mcp` JSON-RPC endpoint for MCP tools.
- `GET /tasks` Lists task definitions.
- `GET /tasks/{task_id}` Details for one task.
- `POST /grader` Grades the current episode without ending it.
- `POST /baseline` Runs the deterministic baseline agent.
- `GET /api/alerts` Lists current alerts.
- `GET /api/alerts/{alert_id}` Returns one alert plus investigation state.
- `GET /threat-intel/ip/{ip}`
- `GET /threat-intel/domain/{domain}`
- `GET /threat-intel/hash/{file_hash}`
- `GET /logs/{source}`
- `GET /ui` Browser dashboard.
- `GET /` Root metadata.

9. MCP Support

The project also exposes the environment through `POST /mcp`.

The implemented tools include:

- `reset`
- `step`
- `state`
- `enrich_indicator`
- `query_logs`
- `correlate_alerts`
- `classify_alert`
- `map_technique`

- `recommend_action`
- `check_asset`
- `check_user`
- `submit_investigation`

This matters because it makes the environment more agent-friendly and easier to integrate with MCP-capable systems.

10. Environment Lifecycle

The environment lifecycle is:

1. `reset(task_id, seed)` Generates a deterministic scenario from the registry.
2. Initializes one `InvestigationState` per alert.
3. Returns an initial `SOCObservation`.
4. Every `step` increments the step counter.
5. Actions are dispatched by type.
6. Step reward is added to cumulative reward.
7. Budget exhaustion auto-ends the episode and applies penalties.
8. `submit_investigation` grades and finalizes the episode.

There is also loop detection:

- repeated identical actions are penalized
- the penalty scales up with repetition

11. Action Handling Logic

`server/environment.py` is the behavioral center of the project.

`enrich_indicator`

- requires `indicator` and `indicator_type`

- infers alert context if `query_alert_id` is absent
- writes the result into `inv.enriched_indicators`
- appends reward bookkeeping and evidence timeline

`query_logs`

- requires `log_source`
- uses `query_alert_id` or defaults to the first alert
- writes entries into `inv.queried_sources`
- appends reward bookkeeping and timeline

`correlate_alerts`

- requires `alert_id_a` and `alert_id_b`
- uses correlation logic from `tools/correlation.py`
- stores the correlation event in both relevant investigations if found
- avoids duplicate correlation storage

`check_asset`

- requires `hostname`
- tries to locate the most relevant investigation
- stores `AssetInfo` in `inv.assets_looked_up`

`check_user`

- requires `username`
- tries to locate the most relevant investigation
- stores `UserInfo` in `inv.users_looked_up`

`classify_alert`

- requires `alert_id` and `classification`

- penalizes classification with no evidence
- counts enrichment, log query, and correlation actions as evidence
- stores `AlertClassification` in `inv.classification`

`map_technique`

- requires `alert_id` and `technique_id`
- penalizes mapping with no evidence
- stores `technique_id` in `inv.mapped_techniques`

`recommend_action`

- requires `alert_id` and `action_type`
- penalizes recommendation with no evidence
- stores `ResponseActionType` in `inv.recommended_actions`

`submit_investigation`

- requires `alert_id`
- ends the episode if all alerts are submitted
- grades the investigation

12. Reward Structure

- `+1.0` for correct classification
- `-1.0` for incorrect classification
- `+0.5` for correct technique mapping
- `-0.5` for incorrect technique mapping
- `+0.2` for correct response action
- `-0.2` for incorrect response action
- `+0.1` for each piece of evidence (enrichment, log query, correlation)

- `-0.1` for each step taken (time budget)
- `-0.2` for each repeated identical action
- `-0.5` for each unsubmitted alert at episode end

13. Time Budget

Each episode has a maximum number of steps. Exceeding this budget ends the episode and applies penalties for unsubmitted alerts.

14. Deterministic Scenario Generation

`scenarios/*.py`

- uses a `seed` to generate the same scenario every time
- ensures reproducibility for benchmarking and debugging
- each scenario defines its own ground truth and backing databases

15. OpenEnv Compliance

- the environment adheres to the OpenEnv standard
- this allows easy integration with other OpenEnv-compatible agents and platforms
- simplifies validation and deployment

16. Task Breakdown

Phishing

This is the easiest task:

- `1` alert total
- `1` true positive

- max steps: `10`

Lateral Movement

This is a medium difficulty task:

- `5` alerts total
- `5` true positives
- max steps: `20`

Queue Management

This is a hard task:

- `20` alerts total
- `10` true positives
- `5` benign true positives
- `5` false positives
- max steps: `50`

Insider Threat

This is the hardest task:

- `30` alerts total
- `9` true positives
- `5` benign true positives
- `16` false positives
- max steps: `80`

Three hidden chains:

- Chain A: unauthorized database access bulk data export suspicious cloud upload
- Chain B: vendor VPN anomaly service account abuse / privilege escalation firewall/configuration changes

- Chain C: after-hours access mass file deletion USB exfiltration

This task is the strongest differentiator for the project because it feels like a true “expert mode” queue.

18. Grading Philosophy

Graders are task-specific and built on shared helpers in `graders/base.py`.

Shared helper capabilities:

- classification accuracy
- technique accuracy
- evidence completeness
- response quality
- efficiency score
- score clamping

Phishing grader

Focus areas:

- correct classification
- technique mapping
- evidence completeness
- response quality

Lateral movement grader

Focus areas:

- all-five classification accuracy
- technique mapping
- kill chain reconstruction
- response quality

- efficiency

Queue management grader

Focus areas:

- F1 over positive-vs-negative classification
- attack-chain recovery
- missed true-positive penalty
- efficiency
- response quality

Insider threat grader

Focus areas:

- F1
- attack chains found
- true-positive coverage
- efficiency
- response quality

The grading design is one of the strongest parts of the project because it makes “good agent behavior” explicit and measurable.

19. Deterministic Baseline Agent

File: `baseline_agent.py`

The current baseline is a deterministic rule-based SOC analyst.

Important facts:

- used by `/baseline`
- reused by `inference.py` when no LLM credentials are configured
- intentionally deterministic

- task-aware

Strategy by task:

- 5-alert tasks: uses ordered kill-chain logic for lateral movement
- large-queue tasks: shifts toward queue-wide classification and response completion
- smaller tasks: enriches, queries logs, correlates, classifies, maps, recommends, submits

Important internal heuristics:

- keyword-based true-positive / false-positive / benign true-positive inference
- task-specific response mapping
- MITRE technique inference from alert titles and sources
- correlation tracking to avoid repeating the same pair forever

This baseline matters for the hackathon because it proves the environment is both runnable and nontrivial.

20. Inference Runner

File: `inference.py`

This script can run:

- with an LLM through OpenAI-compatible APIs
- or with the deterministic baseline fallback

Environment variables used:

- `API_BASE_URL`
- `MODEL_NAME`
- `HF_TOKEN`
- `SERVER_URL`
- `LOCAL_IMAGE_NAME`

- `OPENAI_API_KEY`
- `API_KEY`
- `TASK_TIMEOUT_SECONDS`
- `SEED`

What it does:

- optionally starts the server if it is not already running
- runs all tasks
- logs structured output
- supports a heuristic path when no API credentials are available

21. Benchmark Runner

File: `benchmark.py`

Purpose:

- run `/baseline` repeatedly across tasks and seeds
- check reproducibility
- print a markdown table of results

Current benchmark task list:

- phishing
- lateral_movement
- queue_management
- insider_threat

This is useful for the hackathon because judges care about reproducibility, not just point-in-time demos.

22. Testing

Main test groups:

- API tests
- environment tests
- scenario tests
- grader tests

What they cover:

- endpoint correctness
- reset/step/state behavior
- deterministic generation
- ground-truth completeness
- grader expectations
- regression around bugs like the baseline endpoint and evidence counting

At the time of the latest validation in this repo:

- targeted tests passed
- local OpenEnv validation passed

23. Deployment

The project supports:

- local `uvicorn`
- Docker deployment
- Hugging Face Spaces
- OpenEnv validation

The current HF Space remote is already configured and has been updated.

24. Strengths Of The Project

If you are pitching the project verbally, these are the strongest points:

- realistic domain with clear practical importance

- multiple tasks instead of a single benchmark toy
- deterministic seeded generation
- explicit grading and measurable outcomes
- OpenEnv compatibility
- good baseline and reproducibility story

25. Weaknesses Or Risks

You should know the weaknesses too.

- the project is strongest technically, but still benefits from demo polish
- some of the baseline logic is heuristic and title-driven rather than learned
- the hardest tasks are impressive, but explaining them clearly matters
- external judges may compare it to more visually polished submissions

26. What To Say If A Judge Asks “Why Could This Win?”

A strong answer:

“Because it is not just an app. It is a full agent environment with hidden state, deterministic generation, realistic tools, explicit scoring, multiple difficulty tiers, reproducible baselines, OpenEnv validation, and a domain that actually matters. It benchmarks interactive investigation behavior, not just one-shot prediction.”

27. What To Show In A Demo

Best short demo flow:

1. open the UI or hit `/tasks`
2. show the four tasks and increasing difficulty
3. run `/baseline` on one medium or hard task
4. show the returned score and breakdown
5. explain that the same seed reproduces the same scenario

6. show `benchmark.py` output to prove reproducibility
7. mention that graders inspect evidence, techniques, response, and efficiency

28. Suggested Demo Order

If you only have 2-3 minutes:

- explain the problem
- show the four tasks
- show one live `/baseline`
- show benchmark output
- close on reproducibility plus realism

If you have 5 minutes:

- briefly explain architecture
- show a phishing run
- show one queue task score
- explain grading
- mention OpenEnv validation

29. Suggested Submission Positioning

How to position the project:

- not “a cybersecurity chatbot”
- not “a simple detection benchmark”
- instead: “a training and evaluation environment for sequential SOC investigation”

That wording matters because it places the project in the environment/benchmark category rather than the app/demo category.

30. Final Cheat Sheet

If you forget everything else, remember these:

- environment name: `soc-triage-gym`
- task count: `4`
- hardest task: `insider_threat`
- baseline: deterministic heuristic
- benchmark: deterministic and multi-task
- validation: OpenEnv passes locally
- biggest selling point: realistic interactive investigation workflow
- biggest improvement path: stronger polished demo and multi-seed benchmark presentation

31. Final Recommendation

For the hackathon, this project should be presented as a serious benchmark and agent training environment. Its strongest advantages are realism, structure, determinism, grading, and task diversity. To maximize judging impact, pair the technical strengths with a clean demo, a clear pitch, and benchmark evidence.